

HEINRICH-HEINE-UNIVERSITÄT DÜSSELDORF

Bachelor Thesis

Fast running robotic dog using locomotion policies

Jonas Friesen

Degree:	B.Sc. Computer Science
Supervisor:	Professor Dr. Paul Swoboda
Co-Supervisor:	Dr. Peter Arndt
Faculty:	Faculty of Mathematics and Natural Sciences

July 13, 2026

Abstract

This study investigates the development of high-velocity locomotion policies for the Unitree Go2 quadrupedal robot within the Genesis multi-physics simulation platform. The potential of the Genesis' GPU-native architecture, which enables simulation speeds of up to 430,000 times faster than real time, was explored. Proximal Policy Optimization (PPO) and curriculum learning were employed to teach the robot movement patterns. The generated height-field and fractal terrains taught the robot to navigate different terrain types. The findings of this study demonstrate an exceptional degree of agility. The Unitree Go2 achieved a peak velocity of 8 m/s on a flat terrain, which significantly exceeding traditional trotting speeds and pushing the simulator to its limits. On diverse and unstructured terrain, the robot demonstrated a maximum velocity of 4.5 m/s, substantially outperforming Emre Gökçek's baseline of 1 m/s on the same platform. Furthermore, the goal-conditioned policy enables the robot to execute running curves on command by updating target velocity vectors in real time, while maintaining a top speed of 8 m/s on flat terrain and 4.5 m/s on diverse complex terrain, with a commanded angular velocity of 0.8 rad/s. These results demonstrate the effectiveness of the Genesis simulator in connecting the simulation with high-performance robotic applications, thus providing an efficient platform for high-speed, command-conditioned quadrupedal locomotion in diverse environments.

1. Introduction

Embodied AI is a rapidly evolving field that combines deep reinforcement learning (RL) and computer vision with physical hardware, enabling robots to interact with the real world. One of the challenging problems in robotics training is high speed locomotion. This work aims to push the operational and physical limits of the Unitree Go2 quadrupedal robot by training it within the Genesis physics simulator [1] and utilizing the Proximal Policy Optimization (PPO) [9] algorithm. The main focus is on:

- Command-conditioned policy to train the robot to run at different velocities and in different directions.
- The use of terrain generation, including height fields, fractal terrains and pyramid stairs terrains, to teach the robot to balance on uneven surfaces, an area in which flat-ground training often fails.

First, the work will be contextualized by listing previous studies, followed by an explanation of the methods used to achieve the stated goal. The experiments will then be presented, concluding with a discussion of the results.

2. Related Work

This section provides an overview of the existing literature addressing this problem and the work upon which this study is built, as well as frameworks concerning RL methodologies.

Simulation Platforms. The physics simulation platform Genesis [1] was developed specifically for embodied AI and robotics. Using the Taichi backend [11] for native GPU computations, it can achieve simulation speeds of up to 430 000 times faster than real time. Genesis integrates various physics solvers, such as rigid bodies, fluids and deformable materials into a unified framework, offering a powerful photorealistic rendering system. The platform supports massive parallelization, where over 32 000 environments can be computed simultaneously. Other physics simulators commonly used in robotics research include Isaac Gym [6], MuJoCo [12] and PyBullet [2], all of which have been applied for legged robot training. We chose Genesis due to its massive parallelization throughput, which enables the large-scale parallel training required for high-speed locomotion without requiring a distributed compute cluster.

Reinforcement Learning for Locomotion. John Schulman et al. [9] provide PPO, a new family of policy gradient methods for RL. PPO alternates between sampling data through interaction with the environment and optimizing an objective function using stochastic gradient ascent. While standard policy gradient methods perform only one gradient per update per sample, PPO introduces a new objective function that enables multiple epochs of minibatch updates. PPO has therefore become the standard algorithm for locomotion policy training. Alternative algorithms include Soft Actor-Critic (SAC) [4] and Trust Region Policy Optimization (TRPO) [8], though PPO is generally preferred due to its massively parallel simulation efficiency.

High-Speed Legged Locomotion. Rudin et. al. [6] demonstrate that massively parallel simulation in Isaac Gym enables locomotion policies to be learned in minutes instead of hours establishing the paradigm of large-scale parallel RL-training for quadrupeds upon which this work builds.

Baseline. Emre Gökçek [3] has implemented a successful locomotion policy for the Go2 on fractal terrain, achieving a top speed of 1 m/s. This work serves as the baseline and starting point for the present study. We adopt his environment structure, curriculum implementation and reward formulation as our foundation. We then extend these to reach

higher speeds while training the robot on diverse terrains and enabling goal-conditioned curve running.

3. Method

3.1. Robot and simulation setup

The Go2 is a quadrupedal robot with a 12-dimensional action space, corresponding to three controllable joints in each of its four legs: hip, thigh and calf. The robot model is loaded directly into the simulation environment via a Unified Robot Description Format (URDF) [13] file. The simulation is configured with a control frequency of 50 Hz, matching the frequency of the physical robot. An artificial latency of 20 ms was added to mimic the hardware communication lag and make the simulation more realistic. Additionally, a proportional derivative (PD) controller was added to manage low-level joint actuation. The PD controller translates high-level policy outputs into motor torques, as defined by the following formula:

$$\tau = K_p(q_{ref} - q) + K_d(\dot{q}_{ref} - \dot{q}) \quad (1)$$

where τ denotes the joint torque, q and $\dot{q}$ the current joint angle and angular velocity, q_{ref} and $\dot{q}_{ref}$ the target joint angle and velocity, respectively. The proportional gain K_p corrects instantaneous position errors by producing a torque that is proportional to the deviation from the target angle. The derivative gain, K_d , damps the motion by opposing the rate of change of the error, thereby preventing oscillations. These two terms allow the controller to simultaneously track a desired joint trajectory while rejecting disturbances in a physically stable manner.

The simulation scene supports procedurally generated terrain landscapes via the `gs.morphs.Terrain` class from Genesis. This class generates height-field based surfaces from a set of sub-terrain types. Each sub-terrain tile is defined by a 2-dimensional height map that is sampled at a fixed horizontal and vertical resolutions, enabling the creation of diverse terrains, such as flat, fractal and pyramid staircase terrain. The height field is loaded into the physics solver and is used for collision detection and contact computation during the training.

3.2. Markov Decision Process

High speed locomotion is typically modeled as a Partially Observable Markov Decision Process (POMDP) [5, 10], defined by the tuple $(\mathcal{S}, \mathcal{O}, \mathcal{A}, \mathcal{P}, r, \gamma)$. The robot acts as an agent interacting with the Genesis environment. The goal of the POMDP is to maximize its expected cumulative reward:

- State and observation space $(\mathcal{S}, \mathcal{O})$: While $\mathcal{S}$ represents the physical state of the simulator, the agent operates on an observation space $\mathcal{O}$.

- Action space $(\mathcal{A})$: The action space is 12-dimensional corresponding to the target joint positions.
- Transition Dynamics $(\mathcal{P})$: This is a function $\mathcal{P}(s_{t+1}|s_t, a_t)$, which defines the probability of transitioning to a new state s_{t+1} given the previous state s_t and action a_t .
- Reward function (τ) : The reward function $\tau(s, a)$: A numerical feedback function to determine which state the agent should transition into
- Discount Factor (γ) : A factor used for balancing the importance of present rewards against future ones.

The goal of this process is to get a policy that maximizes the expected discounted return:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \gamma^t r_t \right] \quad (2)$$

3.3. Policy and value functions

The policy and value functions are represented through an actor-critic architecture [10] used for PPO, where the policy function, which is used by the agent to select actions given observations, is defined as $\pi_\theta(a | o)$ and where the value function, which estimates the expected cumulative discounted return from a given state under the current policy, is defined as $V(s)$. Together, they form the basis of the actor-critic [10] framework, in which the policy represents the actor and the value function represents the critic.

Both the actor and the critic are implemented as neural networks (NN), more precisely a Multi-Layer-Perceptron (MLP) with three hidden layers of sizes [512, 256, 128]. Between each layer, an Exponential Linear Unit (ELU) is applied as an activation function to enable non-linearity.

The Actor $\pi_\theta(a|s)$ maps the observation vector to the action space. The network does not predict the absolute motor positions directly, but rather, it predicts the target joint position, multiplied by an action scale of 0.25 and added to a fixed default posture $q_{default}$

$$q_{ref} = 0.25 \cdot a_t + q_{default} \quad (3)$$

This formulation keeps the policy near a stable baseline configuration, which is particularly beneficial in the early stages of training.

The critic network $V(s)$ estimates the expected cumulative discounted return from the current state. The algorithm uses this to judge whether an action was good or bad. An asymmetric actor-critic design is employed, where the actor receives the standard observation vector and the critic additionally receives an additional privileged observation vector containing information about previous joint actions and velocities. This allows the critic to form more accurate value

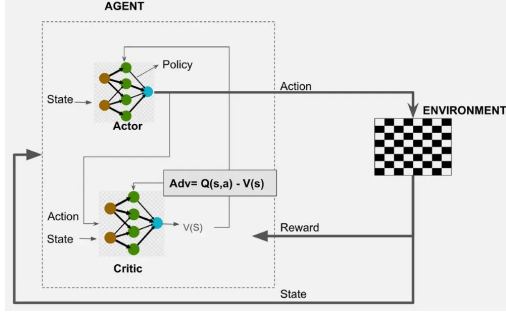


Figure 1. Actor-Critic network visualized

Source: <https://www.geeksforgeeks.org/machine-learning/actor-critic-algorithm-in-reinforcement-learning/>

estimates during training without exposing privileged information to the deployed policy.

Fig. 1 summarizes the interactions between the Actor, Critic and the environment.

3.4. PPO Hyperparameters

The PPO hyperparameters and their values are listed below (Tab. 1). These values are consistent across all experiments discussed in this study.

Hyperparameter	Value
Learning Rate	0.001
Discount Factor (γ)	0.99
GAE Parameter (λ)	0.95
Clip Parameter (ϵ)	0.2
Target KL Divergence	0.01
Number of Mini-batches	4
Number of Learning Epochs	5
Hidden Layer Architecture	[512, 256, 128]
Activation Function	ELU
Action Scale (s_a)	0.25

Table 1. Fixed PPO Hyperparameters across all trainings

Here, the *learning rate* controls the update by how large the steps should be when updating the network.

The *GAE parameter* λ controls the bias-variance trade-off in advantage estimation. A value close to one leads to high variance and low bias that results into relying more on observed long term returns.

The *clip parameter* ϵ represents the core of the PPO algorithm. It limits how much the policy is allowed to change in each update by clipping the probability ratio between the new and old policy.

The *target Kullback-Leibler (KL) divergence* is the difference between the old and the new policy. This hyperparameter stands for a threshold. If the KL divergence exceeds

this threshold, the learning rate is reduced. If the KL divergence stays below the threshold, the learning rate increases. this keeps the policy updates within a safe range. The *number of Mini-Batches* is the collected data split into a specific number of equally sized chunks for each gradient update. This splitting is used for gradient ascent in PPO.

The *number of learning epochs* tells us how many times the same batch is reused for gradient updates before new data is collected. Therefore, increasing this number can lead to overfitting.

3.5. Curriculum Learning

We will use the `rsl_rl`-lib, provided by the Robotic Systems Lab of ETH Zurich [7], for RL training. This library provides an efficient implementation of the PPO algorithm and makes CUDA support available.

We will use a modified version of the library that includes curriculum learning, as implemented by Emre Gökcek [3]. The curriculum changes the maximum commanded velocity $v_x^{\max}$, which is initially set to 1.0 m/s. After each PPO iteration, the mean forward velocity tracking reward, denoted by $\bar{r}_{\text{tracking}}$ is computed over the last 20 iterations. If $\bar{r}_{\text{tracking}}$ exceeds a predefined threshold $\delta_{\text{thresh}} = 0.85$, $v_x^{\max}$ is increased by a fixed value of $\delta_v = 0.05$ m/s:

$$v_x^{\max} \leftarrow v_x^{\max} + \delta_v \quad \text{if } \bar{r}_{\text{tracking}} \geq \delta_{\text{thresh}} \quad (4)$$

Algorithm 1 Velocity Curriculum Update

Require: tracking reward buffer $\mathcal{B}$, current max velocity $v_x^{\max}$, threshold δ_{thresh} , step δ_v , stall counter c

- 1: Append current tracking reward r_{tracking} to $\mathcal{B}$
- 2: Increment stall counter $c \leftarrow c + 1$
- 3: **if** $|\mathcal{B}| = 20$ **then**
- 4: $\bar{r} \leftarrow \frac{1}{20} \sum_i \mathcal{B}_i$
- 5: **if** $\bar{r} \geq \delta_{\text{thresh}}$ **or** $c \geq 500$ **then**
- 6: $v_x^{\max} \leftarrow v_x^{\max} + \delta_v$
- 7: Clear $\mathcal{B}$; reset $c \leftarrow 0$
- 8: **end if**
- 9: **end if**

3.6. Goal-Conditioning

Building on the baseline of Emre Gökcek [3], we transform the fixed commanded target velocity into a range of multiple velocities. This enables the policy to be goal-conditioned. Rather than training on a single target speed, the command vector

$$c = (v_x^*, v_y^*, w_z^*) \in \mathbb{R}^3 \quad (5)$$

is included as a part of the observation at every timestep. This allows the policy to adapt its behavior based on the current command without retraining.

Command Encoding. The command vector c is added to the observation vector and provided as additional information to the actor network at each timestep. This leads to the policy to learn a mapping from observation-command pairs to actions, enabling it to track different velocity and yaw-rate targets within one policy.

Command Resampling. During training, the commanded velocities of c are resampled periodically every four seconds of simulated time for each environment separately. For each dimension, a value is randomly selected from its configuration range:

$$v_x^* \sim \mathcal{U}(v_x^{\min}, v_x^{\max}) \quad (6)$$

$$v_y^* \sim \mathcal{U}(v_y^{\min}, v_y^{\max}) \quad (7)$$

$$\omega_z^* \sim \mathcal{U}(\omega_{\min}, \omega_{\max}) \quad (8)$$

where the ranges for v_x^* and ω_z^* are experiment-specific (Sec. 4). The forward velocity range $[v_x^{\min}, v_x^{\max}]$ is updated by the curriculum mechanism described in Sec. 3.5.

Curve Run Specific. When it comes to curve running, the command vector c (Eq. 5) changes by extending the range of w_z^* from initially $[0.0, 0.0]$ to a new range of $[-0.8, 0.8]$. The values in this range represent the sharpness of the commanded curve, where the values are computed as rad/s. Because we already implemented a resampling method that uniformly samples values within the ranges of each dimension of c , we modify this method to sample not only from v_x^* but also from w_z^* .

A constant yaw rate w_z^* at a given forward velocity v_x^* geometrically defines a circular arc of radius R :

$$R = \frac{v_x^*}{w_z^*} \quad (9)$$

This leads to the robot moving in a smooth circular motion. Another approach would be to give the policy a specific heading direction. But by doing so, the robot would instantly change the direction towards this heading direction. At high speeds, changing the direction so rapidly will cause the robot to collapse, which will reduce its forward velocity significantly.

Motivation. We do this, because when training with a single fixed velocity value, the policy can lead to overfit to that specific command. This results in undesired movements when evaluated at different speeds. By constant resampling of the velocity commands from a given range, the policy is forced to learn a generalized strategy that fits to all different velocities and transitions between gait behaviors. Furthermore, this approach enables real-time control, where the command vector can be updated instantly without any modifications to the policy.

3.7. Code

All of the code and results presented in this work can be found at this Github repository: <https://github.com/jon4s4/Genesis>

4. Experiments

This section will show an overview about the experiments made.

All of the experiments share the following attributes:

- Every PPO Hyperparameter remains the same across all experiments, the exact values are listed in the previous section (Tab. 1).
- The Unitree Go2 has 12 actuated degrees of freedom (DoF), three per leg, hip, thigh and calf joints. The default standing joint angles are listed in Tab. 2.
- The curriculum parameters $curriculum_delta = 0.05$ and $curriculum_threshold = 0.85$ (Sec. 3.5).
- Velocity commands are resampled every four seconds for each environment. A total of 32,768 parallel environments are simulated simultaneously.
- The episode terminates early if one of the following criteria is met:
 1. The robot’s pitch exceeds ± 45 degrees.
 2. The robot’s roll exceeds ± 45 degrees.
 3. One of the robot’s base link contacts the ground.
 4. The robot leaves the terrain boundaries (only on terrain experiments).
- Each episode lasts 20 seconds, equivalent to 1000 steps. When an episode ends, it resets and the robot receives a new randomly sampled velocity.

When training on terrain, we increase the episode length to 40 seconds, to encourage the robot to locomote in a stable way and allow time for more complex tasks, which is necessary for running on complex terrain.

Table 2. Go2 joint default angles (all legs share $k_p = 20$, $k_d = 1.2$).

Joint	FR / FL [rad]	RR / RL [rad]
Hip	0.0	0.0
Thigh	0.8	1.0
Calf	-1.5	-1.5

Additionally, the observation space defined below remains unchanged throughout the experiments.

Observation space. The observation list contains a 51-dimensional vector. We disable the creation of terrain, therefore no height-field data is included. The components of the observation are as follows:

- **Velocity command:** The commanded velocity tells the policy how fast and in which direction the robot should run.
- **Base linear velocity:** This is the robot’s actual velocity at a given moment. We scale this with a value > 1.0 so that differences in velocity are highlighted.
- **Base angular velocity:** This is the body’s rotation velocity. When running, the policy needs to detect the rotation of the robots body to keep balance. Since these rotation impulses can be significant, we scale them by a factor of 0.25.
- **Projected gravity:** The projected gravity vector is a vector perpendicular to the robot’s body. It projects onto the ground. It tells the policy the actual inclination of the robot.
- **Joint positions:** These are the actual joint angles relative to the default pose (Tab. 2).
- **Joint velocities:** These are the actual angular velocities of the joints $\dot{q}_j$. These joint velocities can become numerical large, which is why we scale them by 0.05.
- **Last actions:** These are the joint commands of the previous timestep. This addition to the observation list is crucial, because it gives the policy memory of the last action made.
- **Base displacement:** This is the position change of the body since the last timestep: $\Delta p = p_t - p_{t-1}$. It tells the policy how far the robot has locomoted.

The privileged critic observation has 75 dimensions, which include previous joint velocities and actions. Tab. 3 summarizes all components of the observation and the privileged critic observation:

Evaluation Setup. During the evaluation process, we primarily used the final PPO iteration from each training. We parsed the PPO iteration, the target forward velocity and the target angular velocity as arguments into the evaluation script. We load one environment and let the robot start with a commanded velocity of 0 m/s. Then, we gradually increased the velocity until the given target forward velocity was reached. We then let the robot run until timestep 1000 is reached, which corresponds to a total run time of 20 seconds, is reached. When evaluating on terrain, if the robot reaches the end of the map at timestep 500, it resets back

Table 3. Observation Vector (dim = 75).

Component	Dim	Scale
Velocity command	3	1.0
Base linear velocity	3	2.0
Base angular velocity	3	0.25
Projected gravity	3	1.0
Joint positions	12	1.0
Joint velocities	12	0.05
Actions	12	1.0
Base displacement	3	1.0
Total	51	
<i>Privileged (critic only)</i>		
Previous joint velocities	12	0.05
Previous actions	12	1.0
Total	75	

to the start of the map and starts running again. Our evaluations are visually summarized mainly in three plots. The first plot shows the actual velocity versus the commanded target velocity. The second plot shows the trajectory of the robot on to the x-axis. The third plot shows the actual yaw rate versus the commanded yaw rate in deg/s. Furthermore, for each evaluation, two videos are provided, where one shows the robot in bird’s eye view and the other video shows the robot from the side.

Training setup. The policy was trained using $2^{15} = 32,768$ parallel environments and 5,000 PPO iterations. Training on flat terrain took approximately 2 hours and 30 minutes, while training on diverse terrain took approximately 3 hours. Both were conducted on a single NVIDIA RTX-5090 GPU.

Visual Material. Graphs shown in this study only represents a few evaluations made. More visual material, including videos, will be provided in the Github repository (Sec. 3.7).

4.1. Flat-Terrain Speed Experiment

The goal of this experiment is to test the upper speed limit of the Unitree Go2 in the simulator. The robot is trained to locomote as fast as possible on perfectly flat terrain while maintaining a straight direction.

4.1.1 Reward Functions

The total reward in each reward at each timestep is:

$$r_t = \sum_i w_i \cdot r_i \cdot \Delta t \quad (10)$$

where w_i is the weight for the i -th reward component and $\Delta t = 0.02$ s (50Hz) is the timestep normalizing the rewards to a per second base, which is necessary to make the rewards independent to the simulation frequency.

Forward Velocity Tracking.

$$r_{\text{tracking_lin_vel_x}} = \exp\left(-\frac{(v_x^* - v_x)^2}{\sigma}\right), \quad \sigma = 0.25 \quad (11)$$

This reward tracks the deviation of the current velocity and the commanded target velocity using a Gaussian kernel function. We divide the Error with $\sigma < 1$ to make small differences participate more to the reward.

Angular Velocity Tracking.

$$r_{\text{tracking_ang_vel}} = \exp\left(-\frac{|\omega_z^* - \omega_z|}{\sigma}\right), \quad \sigma = 0.25 \quad (12)$$

The angular velocity tracking penalizes deviations from the target yaw-rate. We decided to use the absolute difference $|\omega_z^* - \omega_z|$ instead of the quadratic difference to emphasize smaller angular errors.

Vertical Velocity Penalty.

$$r_{\text{lin_vel_z}} = -v_z^2 \quad (13)$$

We want to minimize unnecessary energy usage by penalizing the robot's movement in the y-axes with this and motivate the robot to move in no other direction than the x-axes.

Base height Penalty.

$$r_{\text{base_height}} = (p_z - h_{\text{target}})^2 \quad (14)$$

where p_z represents the actual robots base position in the z-axes and h_{target} is the predefined target value of the robots base height. We massively punish any deviation of h_{target} to force the robot to stay in a fixed height position. We set $h_{\text{target}} = 0.28$ which is slightly lower than the Go2's standard height so that the robot has a better position when it comes to run high velocities.

Action Rate Penalty.

$$r_{\text{action_rate}} = -\sum_{j=1}^{12} (a_j^t - a_j^{t-1})^2 \quad (15)$$

The action rate penalty is used to penalize massive changes from one joint command to the previous one to reduce oscillation.

Similar to Default Penalty.

$$r_{\text{similar_to_default}} = -\sum_{j=1}^{12} |q_j - q_j^{\text{default}}| \quad (16)$$

Similar to default penalizes great deviations from the default joint positions. Without that, the robot can learn unnatural joint angles, which can affect the sim-to-real transfer negatively.

Heading Penalty.

$$r_{\text{heading}} = -|\psi|, \quad \psi = \text{yaw} \cdot \frac{\pi}{180} \quad (17)$$

This function penalizes deviations from the straight heading. Where ψ is the yaw angle in radiant relative to the starting orientation.

All the rewards and its predefined weights are summarized in the Tab. 4:

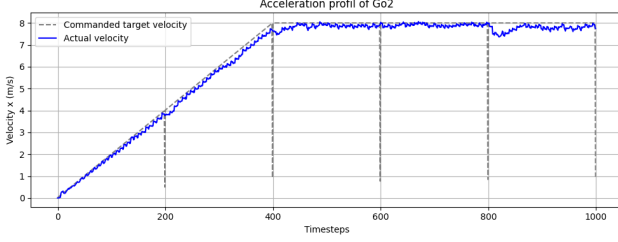
Table 4. Reward terms and weights, flat-terrain experiment.

Reward Term	Weight w_i
tracking_lin_vel_x	+1.0
tracking_ang_vel	+0.5
lin_vel_z	-1.0
base_height	-50.0
action_rate	-0.005
similar_to_default	-0.1
heading	-10.0

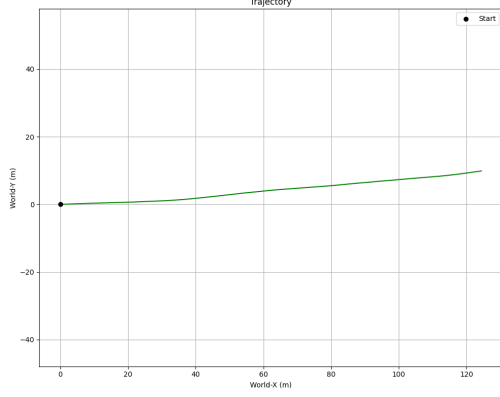
4.1.2 Results and analysis

The policy successfully scales through the velocity curriculum, achieving a maximum forward velocity of 8 m/s on a perfectly flat ground without collapsing (Fig. 2a). It can also run different velocities within the range of [0.5, 8.0]. We evaluated the robot by starting at a low velocity and linearly increasing it until the commanded target velocity was reached. At low velocities, the robot's yaw rate remains stable. However, with increased velocity, its yaw rate breaks out (Fig. 2c). Due to the yaw drift, the robot has difficulty running in a straight line and drifts slightly sideways over time (Fig. 2b). An optimized reward configuration with a stricter penalty for deviation from the initial position on the x-axes could mitigate this problem.

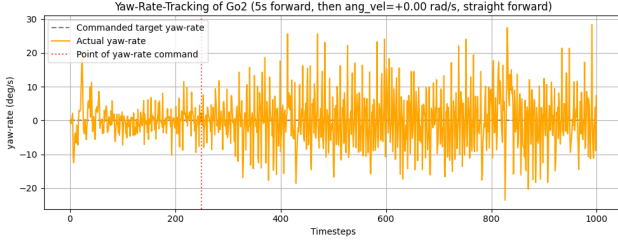
Velocity Curriculum. Our result showed the importance of introducing velocity curriculum learning. Training the robot directly at high end velocities leads to instability and premature termination of the episode. Starting at a lower



(a) Velocity tracking of Go2 at PPO iteration 5000 reaching 8 m/s



(b) Trajectory tracking of Go2 at PPO iteration 5000 reaching 8 m/s



(c) Yaw-rate tracking of Go2 at PPO iteration 5000 reaching 8 m/s

Figure 2. Evaluation results of the flat terrain experiment at PPO iteration 5000, commanded velocity 8m/s

range such as $[0.5, 1.0]$ and incrementally increasing the upper end of the range, the policy is able to learn the locomotion in a stable way. Each curriculum update provides a foundation for the next one, allowing the change of its gait step by step instead of trying to run 8 m/s from scratch.

Reward Design. The driving reward $r_{\text{tracking_lin_vel_x}}$ with its Gaussian kernel function and $\sigma = 0.25$ gives a smooth gradient signal throughout the curriculum learning. Even when the actual velocity differed significantly from the target velocity, the agent received a valuable learning signal.

The r_{heading} penalty was used at high speeds. In earlier iterations of the experiment, we used a lateral displacement instead of the heading penalty. We replaced it with a direct yaw angle penalty to encourage running a straighter line. However, getting fully rid of the yaw drifting was not

achieved in this experiment, where it is difficult to eliminate any yaw drifting when the robots reaches velocities up to 8 m/s. To tackle this problem, hyperparameter tuning is necessary.

Simulation Setup. We increased the number of substeps per control step, because at high speeds, the computations of the rewards can become numerical unstable, which destabilizes the policy.

4.2. Multi-Terrain Experiment

4.2.1 Terrain setup

The goal of this experiment is to build on the previous experiment by having the robot run as fast as possible on different terrains-fractal, pyramid stair and flat-while maintaining a straight direction.

The environment consists of a structured 3×3 grid of sub-terrains, each of which is $12 \text{ m} \times 12 \text{ m}$ and includes flat, fractal and pyramid stairs terrain. The height field has a horizontal resolution of 0.25 meters and a vertical scale of 0.05 meters per unit.

4.2.2 Reward functions

The reward structure and its values are similar to those of Emre Gökçek’s experiment, in which he trained a locomotion policy on uneven terrain [3]. The total rewards used are shown in Tab. 5:

Table 5. Reward terms and weights, multi-terrain experiment.

Reward Term	Weight w_i
tracking_lin_vel_x	+2.0
tracking_ang_vel	+1.0
lin_vel_z	-1.0
lin_vel_y	-5.0
action_rate	-0.005
similar_to_default	-0.1
sideway_movement	-1.0

where $r_{\text{tracking_lin_vel_x}}$, $r_{\text{tracking_ang_vel}}$, $r_{\text{lin_vel_z}}$ and $r_{\text{similar_to_default}}$ are the same as presented previously (Sec. 4.1). Two new formulations are introduced:

Linear y Velocity.

$$r_{\text{lin_vel_y}} = -v_y^2 \quad (18)$$

where this function penalized lateral velocity on the y-axes.

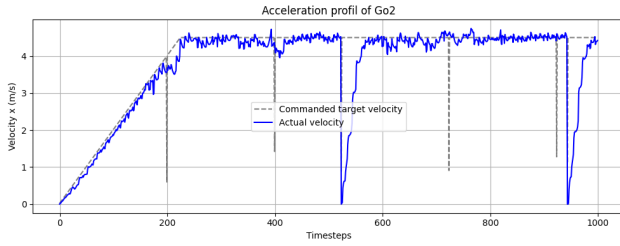
Sideway Movement.

$$r_{\text{sideway_movement}} = -\text{clamp}(|y - y_0|, 0, d_{\text{max}}) \quad (19)$$

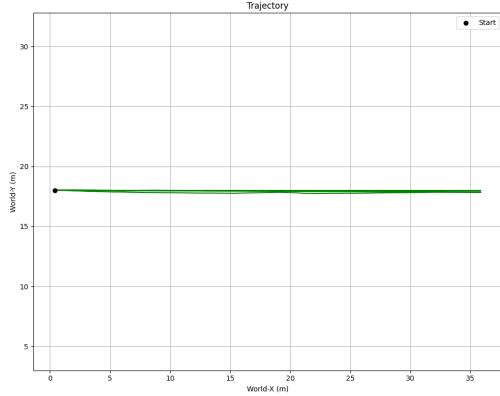
where y is the robot's current lateral position on the y-axis, y_0 is the lateral position at the start of the episode y-axis and d_{max} is a predefined saturation distance. This function penalizes differences in lateral position between the current position and the starting position.

These changes were necessary because, when training on non-flat terrain, we need to allow room for the robot's yaw movement. Without that, the robot cannot locomote over obstacles.

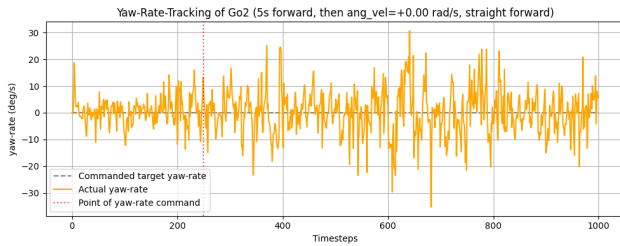
4.2.3 Results and Analysis



(a) Velocity tracking of Go2 at PPO iteration 5000 reaching 4.5 m/s



(b) Trajectory tracking of Go2 at PPO iteration 5000 reaching 4.5 m/s



(c) Yaw-rate tracking of Go2 at PPO iteration 5000 reaching 4.5 m/s

Figure 3. Evaluation results of the multi terrain experiment at PPO iteration 5000, commanded velocity 4.5 m/s

The Go2 successfully learns to locomote across all three

terrain types using a single policy. The robot is able to run up to 4.5 m/s on all three types of terrain, including a combination of them, without collapsing (Fig. 3a). As in the previous experiment, the robot can run at all different velocities within the range of [0.5, 4.5]. This policy successfully trained the robot to locomote through the rough terrain while keeping a straight line (Fig. 3b) and is able to run two times from the one end of the map to the other end of the map within 1000 timesteps without falling.

4.3. High Speed Curve Running on Flat Terrain

This experiment extends the flat terrain speed experiment (Sec. 4.1). The goal is to add a yaw-rate command that requires the robot to execute circular arcs on demand while maintaining high velocities.

4.3.1 Reward functions

The reward function build on the flat terrain formulation (Sec. 4.1). We increase the $r_{\text{tracking_ang_vel}}$ reward from 0.5 to 1.2 because we want to penalize differences between the commanded and the actual angular velocity more severely when the robot was running curves than in the first experiment, in which the goal was to run as fast as possible in a straight line. We additionally replace the r_{heading} penalty with two new formulations:

Heading progress. This reward projects the per-step displacement onto the robot's actual heading position:

$$r_{\text{progress}} = \min(\Delta p \cdot \hat{h}, 1.0) \quad (20)$$

where $\Delta p = p_t - p_{t-1}$ is the two dimensional base displacement and $\hat{h} = (\cos \psi, \sin \psi)$ is the unit heading vector derived from the current yaw angle ψ

Lateral Drift Penalty. We penalize undesired lateral movement of the body's frame, but only when the commanded yaw-rate $|w_z^*| < 0.05$ rad/s. Otherwise no penalty is applied. This results in the formulation:

$$r_{\text{lateral_drift}} = -v_y^2 \cdot 1[|w_z^*| < 0.05] \quad (21)$$

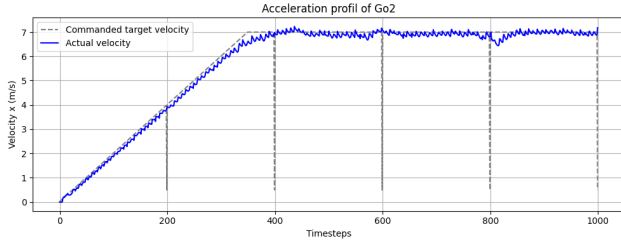
The total rewards used for the training are listed in Tab. 6:

4.3.2 Results and Analysis

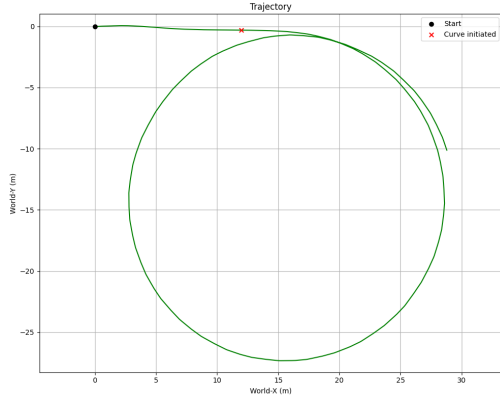
The policy successfully learns both high speed forward locomotion, as well as running smooth right and left curves when a command is given (Fig. 4a and 4b). If no curve command is given, the robot achieves a top speed of 8 m/s (Sec. 3.7), maintaining the maximum velocity of the previous experiment Fig. 2a despite the reward changes. At a yaw-rate command of $w_z^* = 0.8$ rad/s, the robot is able

Table 6. Reward terms and weights, curve experiment.

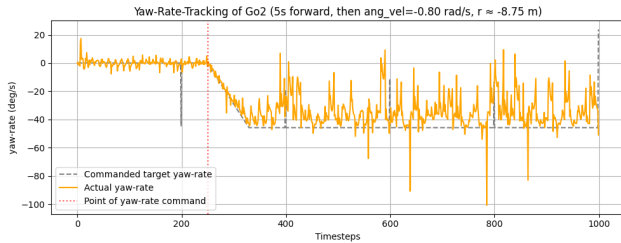
Reward Term	Weight w_i
tracking_lin_vel_x	+1.0
tracking_ang_vel	+1.2
lin_vel_z	-1.0
base.height	-50.0
action.rate	-0.005
similar_to_default	-0.1
progress	+1.5
lateral.drift	-0.3



(a) Velocity tracking of Go2 at PPO iteration 5000 reaching 7 m/s running a right curve



(b) Trajectory tracking of Go2 at PPO iteration 5000 reaching 7 m/s running a right curve



(c) Yaw-rate tracking of Go2 at PPO iteration 5000 reaching 7 m/s running a right curve

Figure 4. Evaluation results of the curve running on flat terrain experiment at PPO iteration 5000, commanded velocity 7 m/s

to run a smooth circular motion while sustaining a forward velocity of 7 m/s, corresponding to an arc radius of $R = 7.0/0.8 = 8.75m$.

We proved that the two implementation changes of the rewards were crucial. First, the r_{heading} reward had to be removed. It contradicted the yaw tracking objective by penalizing any difference of the heading direction. Second, the addition of the r_{progress} reward and the $r_{\text{lateral_drift}}$ reward play a complementary role as the progress reward motivates the robot when he is moving forward in the direction he currently is heading and the lateral drift penalty suppresses undesired sideways slips during phases, where the robot has to run in a straight line.

4.3.3 Optimizations

Although we achieved a strong result, there are still optimizations that can be made. Besides the limitations mentioned in the previous experiment (Sec. 4.1), the robot still does not run perfectly straight. To eliminate this problem, it is beneficial to tune the rewards used for this training. Another approach is to, instead of uniformly and randomly sample across the angular range $w_z^* = [-0.8, 0.8]$, we can induce a change that 30% of all commanded angular velocity samples are 0. This shifts the focus from equally training speed and curve running to high speed running in a straight line.

Additionally, the angular velocity range w_z^* can be expanded to run a sharper curve at higher speed. One possible approach is to extend the curriculum learning implementation by increasing not only the maximum commanded target forward velocity v_x^{max} but also the lower and upper bound of w_z^* , where we start with a lower bound (e.g $[-0.5, 0.5]$) and slightly increase it at specific PPO iterations.

4.4. High Speed Curve Running On Different Terrain Types

Just as we expanded the flat terrain experiment, we aim to do the same for the multi terrain experiment (Sec. 4.2). The goal is to train the robot to run curves on complex, diverse terrain while maintaining a high forward velocity.

4.4.1 Terrain setup

The terrain setup is the same as the setup from the multi terrain experiment (Sec. 4.2), where we create for each environment a structured 3×3 grid of sub-terrains, including flat terrain, fractal terrain and pyramid staircases.

4.4.2 Reward Functions

No new unseen reward functions were implemented. We took the reward function from the multi terrain experiment

(Sec. 4.2) and substituted the $r_{\text{sideway_movement}}$ reward with r_{progress} (20) and $r_{\text{lateral_drift}}$ (21).

We increase the $r_{\text{tracking_lin_vel_x}}$ and $r_{\text{tracking_ang_vel}}$ rewards from 1.0 to 2.0. Additionally, we set r_{progress} to 2.0 and $r_{\text{lateral_drift}}$ to -2.0

The total rewards get together (Tab. 7):

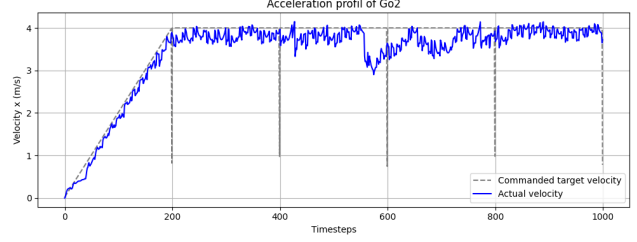
Table 7. Reward terms and weights, curve experiment.

Reward Term	Weight w_i
tracking_lin_vel_x	+2.0
tracking_ang_vel	+2.0
lin_vel_z	-1.0
lin_vel_y	-5.0
action_rate	-0.005
similar_to_default	-0.1
progress	+2.0
lateral_drift	-2.0

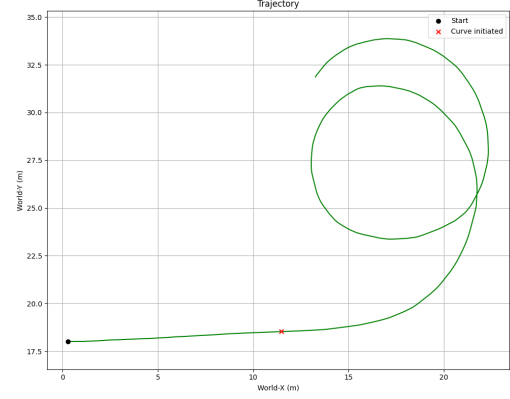
4.4.3 Results and Analysis

Our results demonstrate a successful training, where the robot is able to run a right curve and a left curve with sharpness 0.8 rad/s (Fig. 5b) while maintaining a forward velocity of 4 m/s (Fig. 5a), which corresponds to an arc radius of $R = 4.0/0.8 = 5m$. When no angular command is given, the robot reaches a top speed of approximately 4.5 m/s (Sec. 3.7). Due to the complex terrain and stronger drift from the x-axis, the robot is cannot run a smooth circular motion such as shown in the curve running experiment on flat terrain (Sec. 4.3).

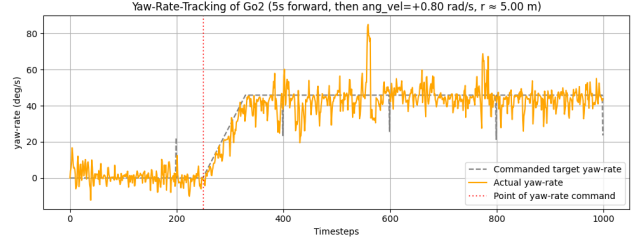
We have tested different variations of reward configurations. We needed to increase both tracking rewards, because otherwise the robot could not learn an efficient gait behavior to reach top speeds, where he stagnated around 2.5 m/s maximum velocity by the end of the training. By increasing only one of the tracking rewards, the robot learns to focus on the tracking reward with the higher weight. Therefore, if we would have only increased $r_{\text{tracking_ang_vel}}$, the robot would successfully learn to run smoother curves, but would only reach a top speed at around 2.5 m/s. To shift the focus slightly more to reaching high speeds, we decided to increase the r_{progress} reward to 2.0, which enables the policy to learn both, running a curve and running at high velocities, appropriately while focusing slightly more on learning to run fast. We tried to tackle the yaw-drifting problem by gradually increasing the $r_{\text{lateral_drift}}$ penalty and ended up with a weight of -2.0 without success.



(a) Velocity tracking of Go2 at PPO iteration 5000 reaching 4 m/s running a left curve



(b) Trajectory tracking of Go2 at PPO iteration 5000 reaching 4 m/s running a left curve



(c) Yaw-rate tracking of Go2 at PPO iteration 5000 reaching 4 m/s running a left curve

Figure 5. Evaluation results of the curve running on diverse terrain experiment at PPO iteration 5000, commanded velocity 4 m/s

4.4.4 Optimizations

Although we successfully implemented a single policy, which learns to run a curve on complex and diverse terrain while maintaining top speeds, the results demonstrate some weaknesses like yaw-drifting and unstable locomotion. When running the evaluation for running curves at 4 m/s or more, the results strongly vary. This shows that the policy is still fragile even when testing within the simulator. To solve this problem, hyperparameter tuning is necessary. Because the gait behavior in comparison to the Multi-Terrain experiment (Sec. 4.2) got worse, it makes sense to split the single policy into two separate ones, where the first one focuses only on the forward locomotion and establish-

ing a solid baseline, while the other one build upon the first policy and learns with the given gait behavior to run smooth curves.

Another approach would be to increase the maximum PPO iterations of the training. We observed that the mean action noise standard deviation, which represents the mean standard deviation across all 12 joints, by the end of the training is higher than in the previous experiments. This metric shows that the policy is still uncertain in which gait behavior it should adapt to. The high standard deviation is mainly caused by an unoptimized reward configuration, where we reward high velocity running and punish behavior which might lead to high velocity running.

4.5. Limitations

Although we achieved strong results throughout the different experiments, some limitations must be named.

No domain randomization was applied. Physical parameters such as ground friction, link masses and motor gains are fixed at their nominal values. This means the policy is optimized for a single perfect environment and will be confronted with problems when transferring this to real hardware.

Also the policy has not been exposed to disturbances like pushes from different sides or uneven surfaces. Future work should therefore include domain randomization and sim-to-real transfer validations to bridge the gap between the observed simulation performance and the transfer to the real world.

5. Conclusion

This study investigated high velocity locomotion policies for the Unitree Go2 quadrupedal robot, trained within the physics simulator Genesis using PPO with forward velocity curriculum learning. Across the four experiments, where the policy achieved a top speed of 8 m/s on flat terrain and up to 5 m/s on complex terrain, the policy shows strong agility and generalization, substantially outperforming the baseline of Emre Gökçek, which this study is built upon.

The goal-conditioned curve-running policy on flat terrain successfully combines high-speed locomotion with dynamic guidance. Our experiments show the importance of the introduced r_{heading} and $r_{\text{later_drift}}$ rewards which replaced the for the curve running counterproductive $r_{\text{sideway_movement}}$ reward.

Three design changes proved to be essential for achieving these results. First, increasing the derivative gain of the PD controller from $K_d = 1.0$ to $K_d = 1.2$ was critical for reaching higher forward velocities. The stronger damping term reduces oscillatory joint behavior at higher speeds. Second, replacing the single fixed target velocity with a continuous command space forced the policy to learn a generalized locomotion strategy which transitions

smoothly between different gaits and to eliminate overfitting on one fixed target velocity. Third, periodically resampling the command vector every 4 seconds of simulated time for each environment ensured that the policy is continuously exposed to the full spectrum of velocities and yaw-rates, which is crucial for command conditioning.

Future work should address the issues that occurred during our experiments. No domain randomization was applied, where physical parameters were fixed the whole time. Furthermore, the robot was not exposed to any outside disturbances. Therefore the robot is not prepared for external forces affecting him.

Finally, the curve-running experiment on terrain revealed that a single trained policy struggles to learn both objectives, running at high speeds and running a curve, equally well. Splitting the training into two sequential stages could improve performance in this setting.

References

- [1] Genesis Authors. Genesis: A universal and generative physics engine for robotics and beyond, December 2024. <https://github.com/Genesis-Embodied-AI/Genesis>. 1
- [2] Erwin Coumans and Yunfei Bai. Pybullet, a python module for physics simulation for robotics, games and machine learning. <http://pybullet.org>, 2021. Abgerufen am: 11. Juli 2026. 1
- [3] Emre Gökçek. Robotic dog walking policies in the genesis simulator. <https://github.com/ahemyu/GenesisLocomotionPolicies>, June 2025. 1, 3, 7
- [4] Tuomas Haarnoja, Aurick Zhou, Kristian Hartikainen, George Tucker, Sehoon Ha, Jie Tan, Vikash Kumar, Henry Zhu, Abhishek Gupta, Pieter Abbeel, and Sergey Levine. Soft actor-critic algorithms and applications. *arXiv preprint arXiv:1812.05905*, 2018. <https://arxiv.org/abs/1812.05905>. 1
- [5] Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1–2):99–134, 1998. <https://www.sciencedirect.com/science/article/pii/S000437029800023X>. 2
- [6] Mayank Mittal, Pascal Roth, James Tigue, Antoine Richard, et al. Isaac lab: A gpu-accelerated simulation framework for multi-modal robot learning. *arXiv preprint arXiv:2511.04831*, 2025. <https://arxiv.org/abs/2511.04831>. 1
- [7] Nikita Rudin, David Hoeller, Philipp Reist, and Marco Hutter. RSL-RL: A fast and simple implementation of RL algorithms. https://github.com/leggedrobotics/rsl_rl, 2021. Legged Robotics, ETH Zurich. 3
- [8] John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pages 1889–1897. PMLR, 2015. <https://arxiv.org/abs/1502.05477>. 1

- [9] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, August 2017. <https://arxiv.org/abs/1707.06347>. 1
- [10] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2018. <http://incompleteideas.net/book/the-book-2nd.html>. 2
- [11] Taichi Graphics Authors. Taichi documentation and back-end reference. <https://docs.taichi-lang.org/>, 2026. 1
- [12] Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5026–5033. IEEE, 2012. <https://ieeexplore.ieee.org/document/6386109>. 1
- [13] Unitree Robotics. Unitree go2 robot description (urdf). https://github.com/unitreerobotics/unitree_ros/tree/master/robots/go2_description/urdf, 2024. 2

Erklärung

Hiermit versichere ich, dass ich die Bachelorarbeit selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Generative KI wurde ausschließlich in den folgenden Bereichen eingesetzt:

- Literaturrecherche
- Generieren der Latex Tabellen, Formeln und Zitierungen (in references.bib)
- Generieren von python Code, welcher für die Experimente genutzt wurde, insbesondere das Erstellen der Methoden in den "reward_wrapper.py" Klassen, und den Änderungen der "go2_env.py" Klassen.
- Textverbesserungen für einzelne Wörter und Rechtschreiboptimierungen

Düsseldorf, den 13.07.2026

Jonas Friesen